Artificial Intelligence — Lecture 4

Constraint Satisfaction Problems

Dr. Fayçal Hamdi

Faculty of Computer and Information Systems

Lecture 4 at a Glance

Constraint Satisfaction Problems

Course Learning Outcome

CLO4: Investigate constraint satisfaction for a given set of problems.

Lecture purpose

- Represent problems using **variables**, **domains**, and **constraints**.
- Explain why CSPs can be solved more efficiently than uninformed search by exploiting structure.
- Study backtracking search, constraint propagation, arc consistency, variable/value heuristics, and local repair.

Conceptual structure

Stage	Core question
Representation	How do we model a problem as variables, domains, and constraints?
Inference	How can constraints reduce the remaining search space?
Search control	Which variable and value should be tried next?
Repair	How can a complete but inconsistent assignment be improved?

💡 Main idea: a CSP separates the **problem structure** from the **search procedure**, allowing the algorithm to reason about constraints before blindly trying assignments.

Learning Objectives

By the end of Lecture 4, you will be able to:

1. Formulate a problem as a **CSP** using **variables**, **domains**, and **constraints**.
2. Distinguish **complete assignments**, **partial assignments**, **consistency**, and **solutions**.
3. Draw and interpret a **constraint graph** for **binary CSPs**.
4. Apply **backtracking search** to a small **CSP**.
5. Explain **forward checking** and distinguish **node consistency**, **arc consistency**, and **path consistency**.
6. Apply **arc consistency** and trace the **AC-3 algorithm** on a small **example**.
7. Apply **MRV** and **Degree** for **variable selection** and **LCV** for **value ordering**.
8. Describe **min-conflicts** as a **local-search** method for **CSPs**.

Lecture Roadmap

From representation to intelligent constraint-based search

Part	Main idea	Student output
1. CSP representation	Problems can be described using variables, domains, and constraints.	Formalize examples such as map coloring, Sudoku, N-Queens, and timetabling.
2. Backtracking and inference	Search can be strengthened by forward checking and progressively stronger consistency reasoning.	Trace backtracking, forward checking, node/arc/path consistency, AC-3, and MAC.
3. Heuristics, repair, and structure	Variable/value ordering and problem structure can substantially change search effort.	Apply MRV, Degree, LCV, min-conflicts, and reason about constraint-graph structure.

Part I

CSP Representation and Constraint Graphs

A CSP asks: **Can we assign values to variables so that all constraints are satisfied?**

Why Constraint Satisfaction?

Many AI problems are naturally constraint-based

Map coloring

Assign a color to each region so that neighboring regions have different colors.

Scheduling

Assign courses, teachers, rooms, and times while avoiding conflicts and capacity violations.

Sudoku

Assign digits to cells so that rows, columns, and subgrids contain each digit once.

N-Queens

Place queens on a board so that no two queens attack each other.

💡 The key advantage is that constraints are explicit. The solver can use them to prune impossible choices early.

CSP Formal Definition

Variables, domains, and constraints

$$CSP = (X, D, C)$$

where:

Component	Meaning	Example
$X = \{X_1, \ldots, X_n\}$	Set of variables	Regions in a map
$D = \{D_1, \ldots, D_n\}$	Domain of possible values for each variable	Available colors
$C = \{C_1, \ldots, C_m\}$	Constraints restricting allowed combinations of values	Adjacent regions must differ

A constraint C_i has a **scope** and a **relation**:

$$C_i = (scope_i, rel_i)$$

where rel_i contains the allowed combinations of values for the variables in $scope_i$.

Assignments and Solutions

CSP search is assignment search

Term	Meaning
Assignment	A mapping from some variables to values.
Partial assignment	Some variables have values, but not all.
Complete assignment	Every variable has a value.
Consistent assignment	No constraint is violated by the assigned variables.
Solution	A complete and consistent assignment.

A CSP solution is an assignment A such that every variable has a value and every constraint in C is satisfied.

Constraint Types

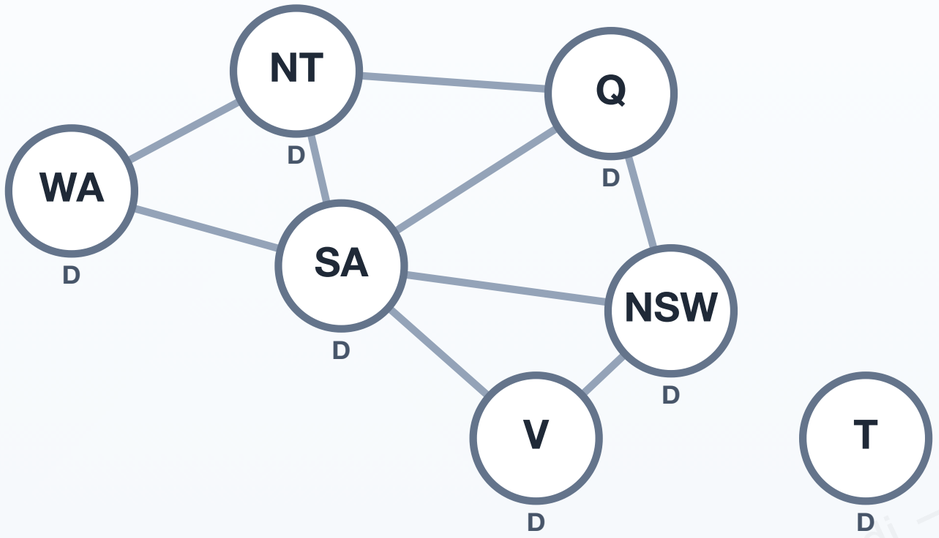
Arity and semantics describe different kinds of constraints

Type	Scope	Example	Main idea
Unary	1 variable	$X \neq 3$	Restricts the domain of one variable directly.
Binary	2 variables	$X \neq Y$	Relates two variables; naturally represented by an edge in a constraint graph.
Higher-order	3 or more variables	$X + Y = Z$	Relates several variables simultaneously.
Global	Usually many variables	$AllDifferent(X_1, \ldots, X_n)$	Represents a recurring semantic relation that a solver may propagate using a specialized algorithm.

💡 **Arity** tells us how many variables are involved. A **global constraint** is more than simply “a constraint with many variables”: it usually carries useful structure that a CSP solver can exploit directly.

Constraint Graph

A visual model for binary CSPs



Interpretation

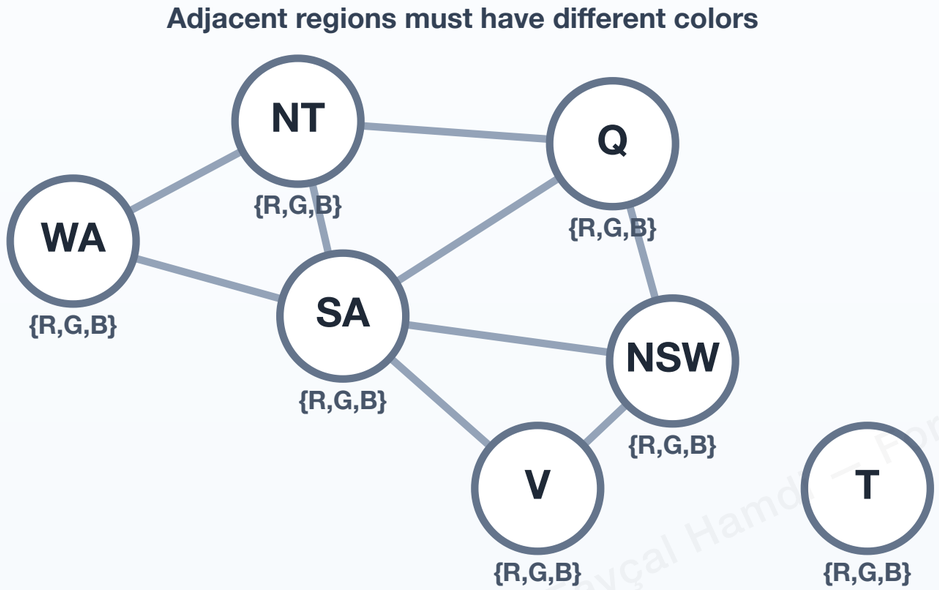
- A node represents a variable.
- An edge represents a binary constraint.
- For map coloring, every edge means “different colors”.
- The graph reveals which choices can influence each other.

Tasmania has no adjacency edge with the mainland regions, so it can often be assigned independently.

💡 A standard constraint graph is most natural for **binary CSPs**. Higher-order or global constraints may be represented by richer graph structures or handled directly by the solver rather than decomposed into ordinary binary edges.

Map Coloring as a CSP

Domain reduction during assignment



Step 1 — CSP model

Main idea
Each region is a variable. Each variable has a domain of possible colors, and edges represent binary inequality constraints.

Variables

WA, NT, SA, Q, NSW, V, T

Domains

Each variable initially has {R, G, B}

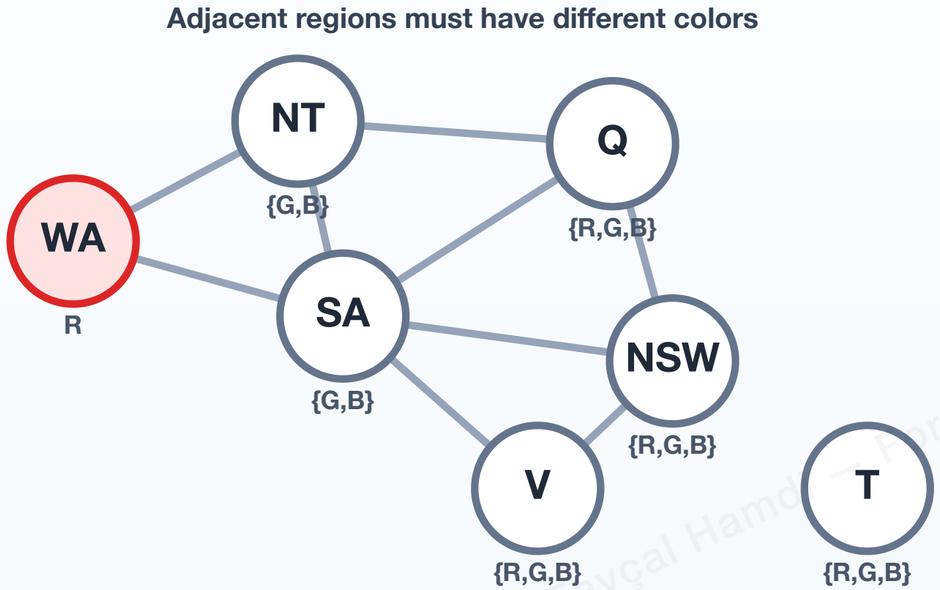
Constraints

Neighboring regions cannot share the same color.

Legend

Assigned: R G B unassigned

Domain reduction during assignment



Step 2 — assign WA

Main idea
Assign WA = Red. Red is removed from the domains of its unassigned neighbors NT and SA.

Assignment

WA = Red

Domain reduction

NT = {G,B}, SA = {G,B}

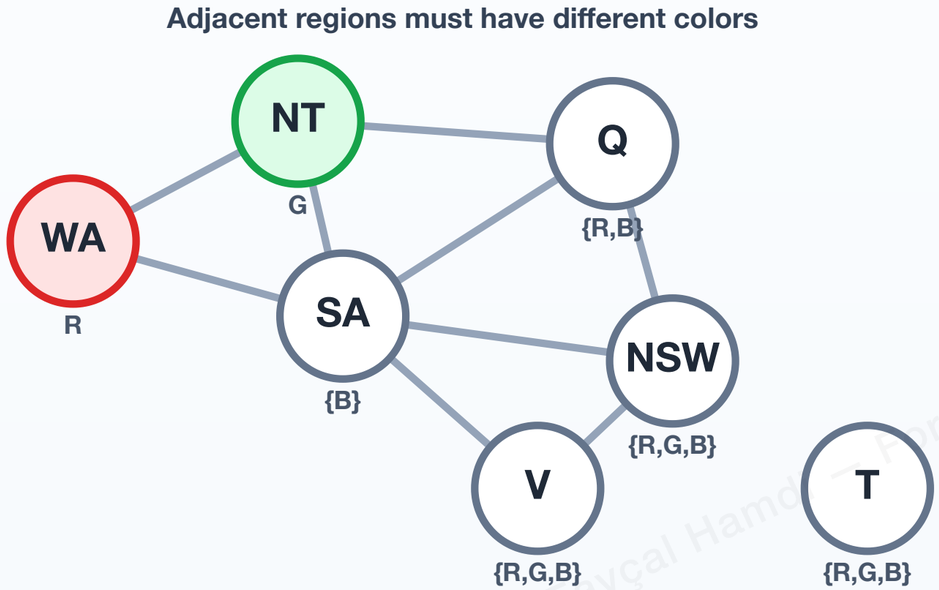
Status

The partial assignment remains consistent.

Legend

Assigned: R G B unassigned

Domain reduction during assignment



Step 3 — assign NT

Main idea
Assign NT = Green. Green is removed from the domains of its unassigned neighbors SA and Q.

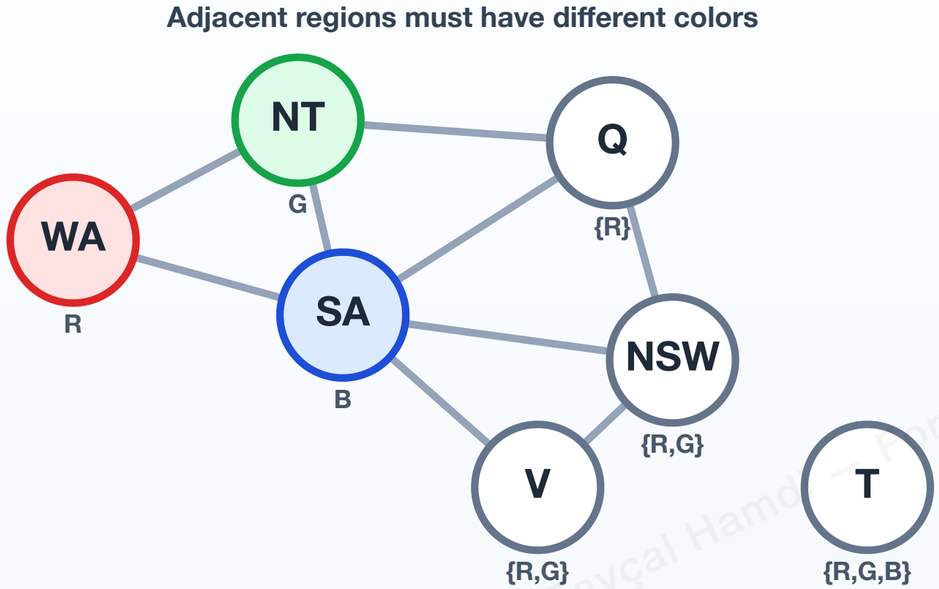
Assignment
WA = Red, NT = Green

Inference
SA = {B}, so SA must be Blue. Q is reduced to {R,B}.

Status
Domain reduction is narrowing the remaining choices.

Legend
Assigned: R G B unassigned

Domain reduction during assignment



Step 4 — assign SA

Main idea
Assign SA = Blue. Blue is removed from the domains of its unassigned neighbors.

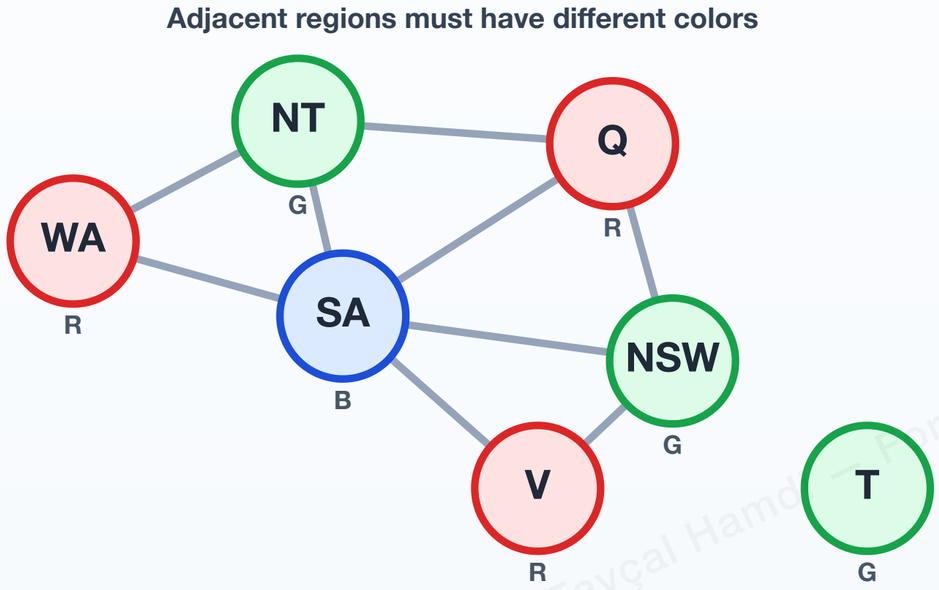
Assignment
SA = Blue

Inference
Q = {R}, so Q must be Red. NSW and V are reduced to {R,G}.

Status
The remaining variables are increasingly constrained.

Legend
Assigned: R G B unassigned

Domain reduction during assignment



Step 5 — complete solution

Main idea
Continue assigning consistent values until every variable is assigned and every constraint is satisfied.

Solution

WA=R, NT=G, SA=B, Q=R, NSW=G, V=R, T=G

Completeness

Every variable has a value.

Consistency

Every adjacency constraint is satisfied.

Legend

Assigned: R G B unassigned

Generate-and-Test versus Constraint Reasoning

Why CSP algorithms do more than enumerate assignments

Method	Behavior	Weakness
Generate-and-test	Generate a complete assignment and then test all constraints.	Wastes effort on assignments that contain early conflicts.
Backtracking	Build a partial assignment and reject it as soon as it violates a constraint.	Still may explore many branches without guidance.
Constraint propagation	Remove values that cannot participate in any solution.	Extra inference costs time but can drastically reduce search.

> CSP algorithms are powerful because they combine **search** with **inference**.

Part II

Backtracking and Constraint Propagation

Backtracking searches over assignments. Propagation tries to detect impossible values **before** committing to deeper choices.

Backtracking Search

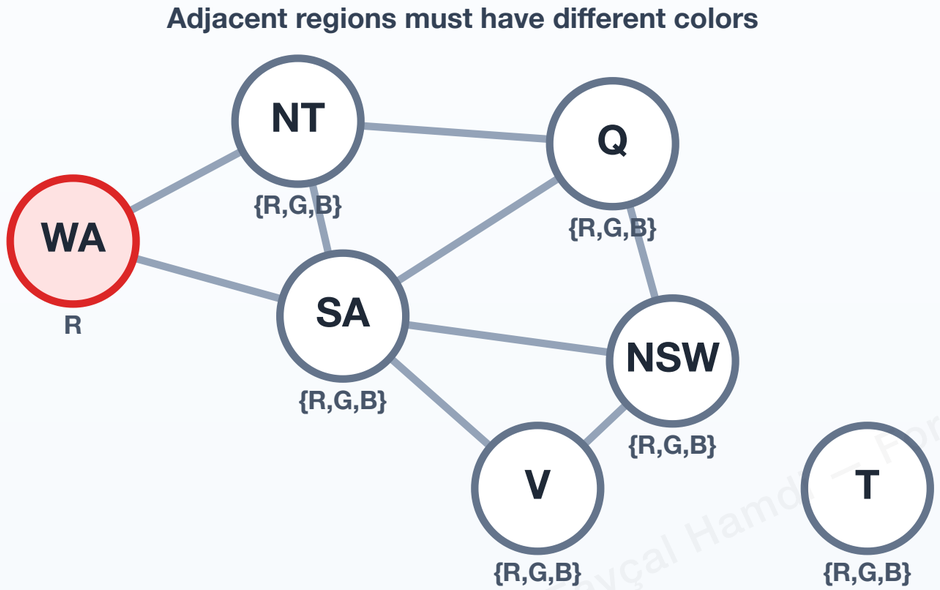
Depth-first search over partial assignments

```
def backtracking_search(csp):
    return backtrack({}, csp)

def backtrack(assignment, csp):
    if assignment is complete:
        return assignment

    var = select_unassigned_variable(csp, assignment)

    for value in order_domain_values(var, csp, assignment):
        if value is consistent with assignment:
            add {var = value} to assignment
            result = backtrack(assignment, csp)
            if result != failure:
                return result
            remove {var = value} from assignment

    return failure
```


💡 The algorithm searches depth-first, but it prunes a branch as soon as the partial assignment becomes inconsistent.

Backtracking on Map Coloring

Rejecting inconsistent branches early



Step 1 — make a partial assignment

Main idea
Start with a consistent partial assignment. So far, only WA has been assigned.

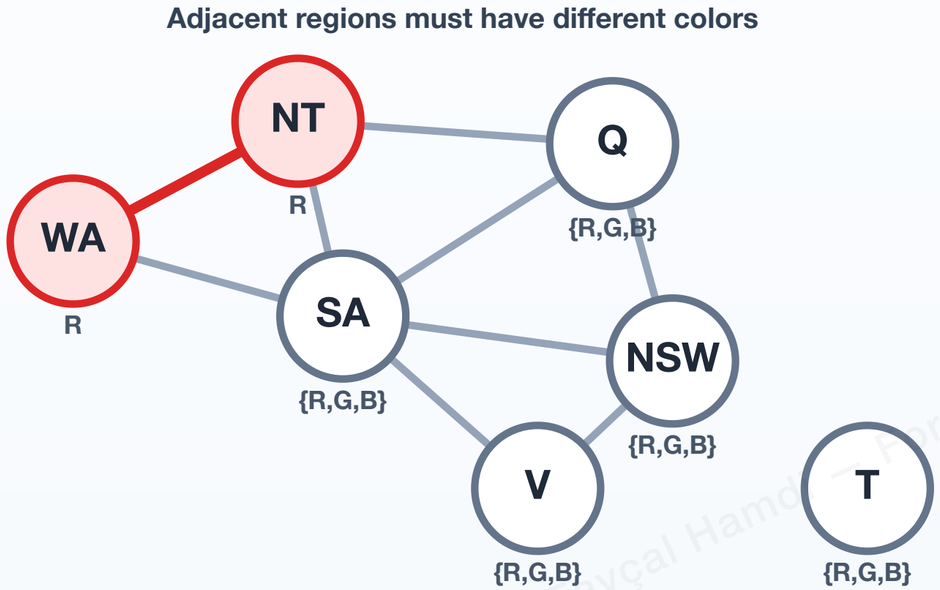
Current assignment
WA = Red

Status
No constraint is violated yet.

Next move
Try a value for the next variable, NT.

Legend
Assigned: R G B ◯ unassigned

Rejecting inconsistent branches early



Step 2 — detect inconsistency early

Main idea
Try NT = Red. The constraint WA ≠ NT is violated immediately, so there is no reason to extend this branch further.

Tentative assignment

WA = Red, NT = Red

Conflict

WA and NT are adjacent but have the same color.

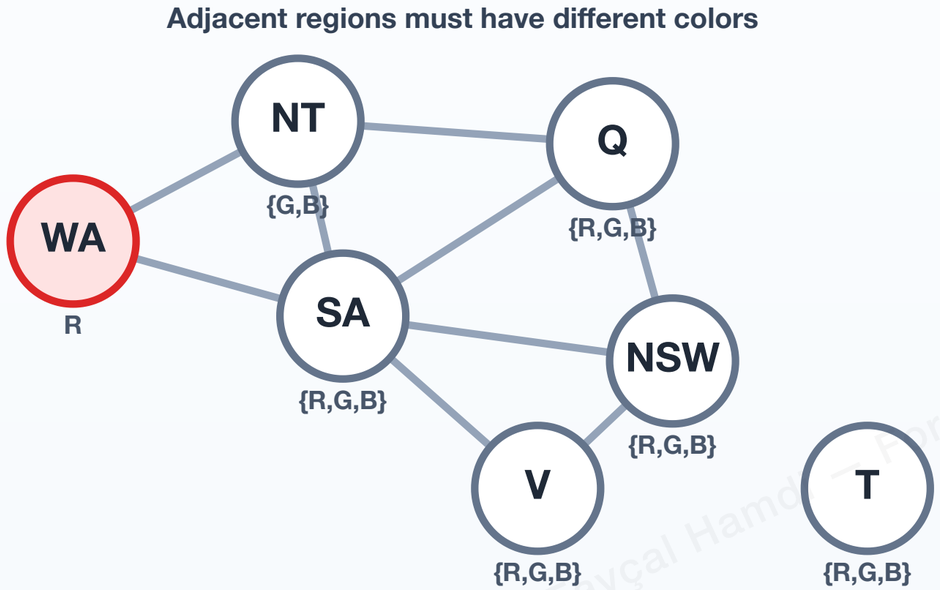
Action

Reject this inconsistent branch immediately.

Legend

Assigned: R G B unassigned

Rejecting inconsistent branches early



Step 3 — reject and backtrack

Main idea
Backtracking undoes NT = Red and returns to NT. Red has already been tried and rejected; Green and Blue remain available to test.

Rejected choice

NT = Red

Untried legal values

Green or Blue

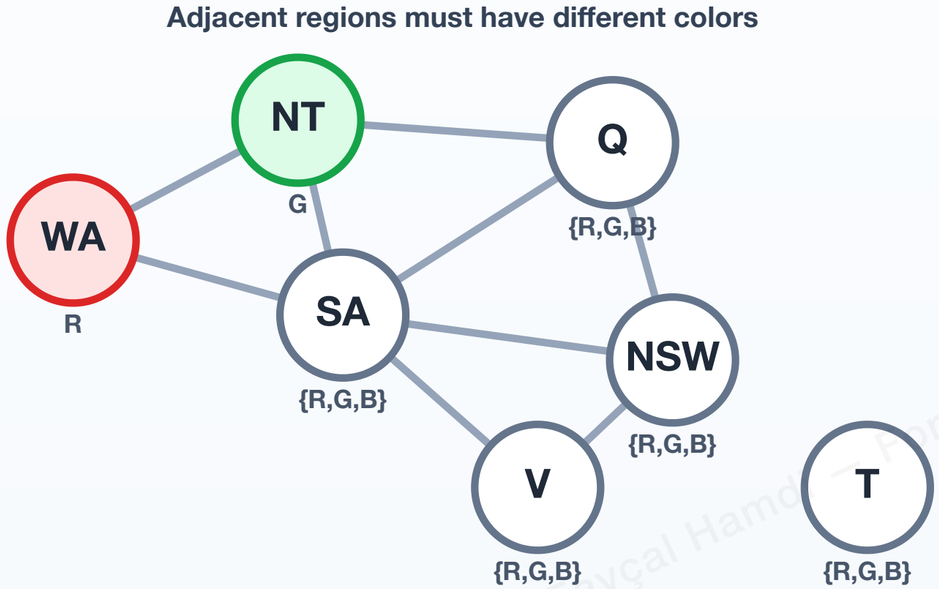
Important distinction

This is search bookkeeping, not propagation to future variables.

Legend

Assigned: R G B unassigned

Rejecting inconsistent branches early



Step 4 — try another value

Main idea
Try NT = Green. Now WA ≠ NT is satisfied, so the search can continue with the next variable.

New assignment
WA = Red, NT = Green

Consistency check
All assigned neighbors have different colors.

Next move
Select another unassigned variable and continue the search.

Legend
Assigned: R G B unassigned

Forward Checking

Propagate only from the variable just assigned

After assigning $X_i = v$, forward checking examines each **unassigned neighbor** X_j and removes every value inconsistent with $X_i = v$.

> Forward checking reports failure immediately if an unassigned neighbor loses **all** values from its domain.

Example

If $WA = Red$, then adjacent variables NT and SA can no longer use Red.

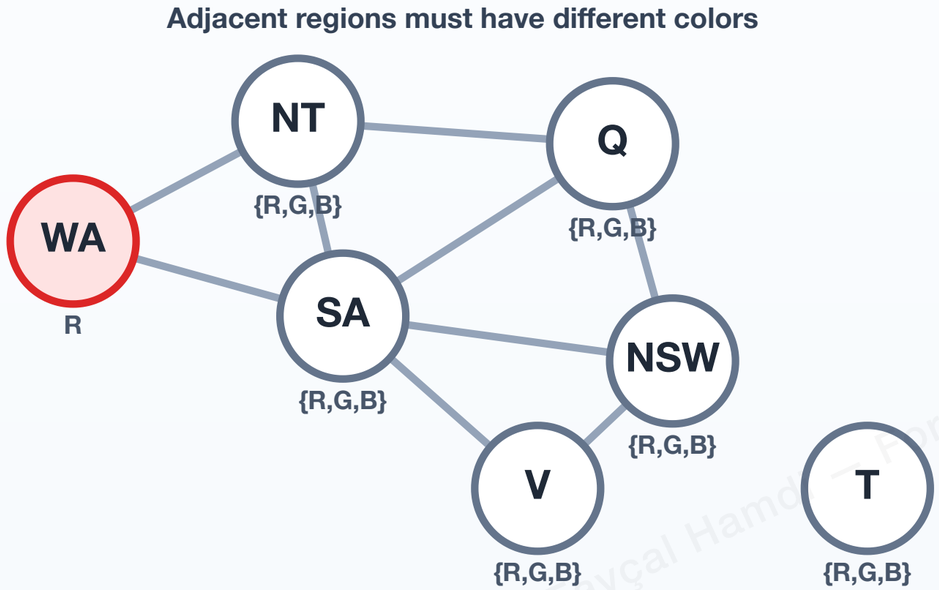
What forward checking does not do

Suppose two variables X and Y are still unassigned, both have domain $\{Red\}$, and the constraint is $X \neq Y$.

- Forward checking may not detect the contradiction yet because neither variable has just been assigned.
- Arc consistency checks the relation between their domains directly and detects that no supporting value exists.

💡 **Bridge:** forward checking reasons from the **latest assignment**; arc consistency can propagate constraints **between unassigned variables as well**.

Forward Checking in Action



Step 1 — assign WA

Main idea
Assign WA = Red. Forward checking now examines the unassigned variables directly constrained by WA.

Assignment

WA = Red

Current domains

All unassigned variables initially have {R,G,B}.

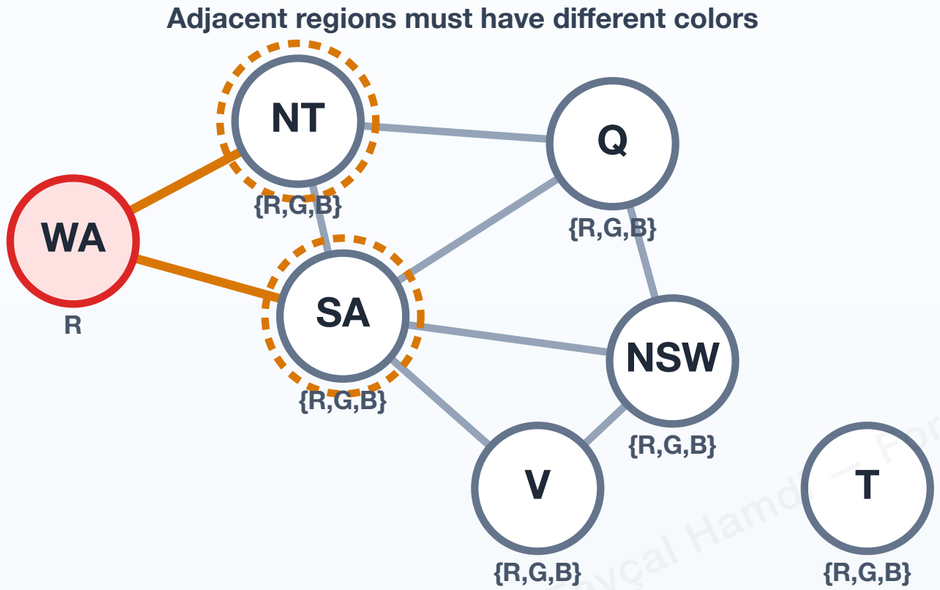
Next action

Check the unassigned neighbors of WA.

Legend

Assigned: R G B unassigned

Forward Checking in Action



Step 2 — inspect unassigned neighbors

Main idea
Only unassigned neighbors directly connected to WA are checked: NT and SA.

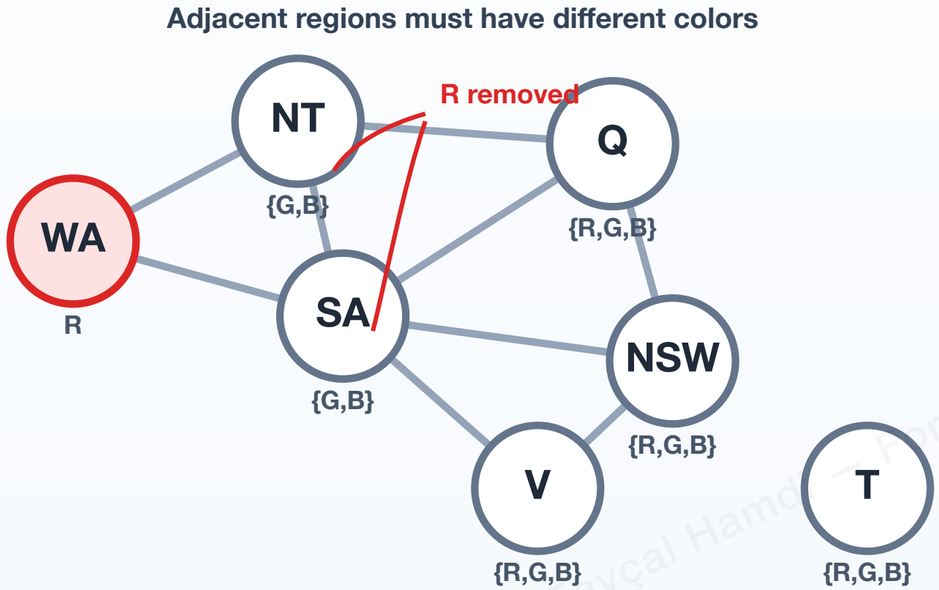
Affected neighbors
NT and SA

Constraint
Neither neighbor may take Red while WA = Red.

Action
Remove values inconsistent with WA = Red.

Legend
Assigned: R G B unassigned

Forward Checking in Action



Step 3 — prune inconsistent values

Main idea
Red is removed from the domains of NT and SA because either assignment would conflict with WA = Red.

NT

{R,G,B} → {G,B}

SA

{R,G,B} → {G,B}

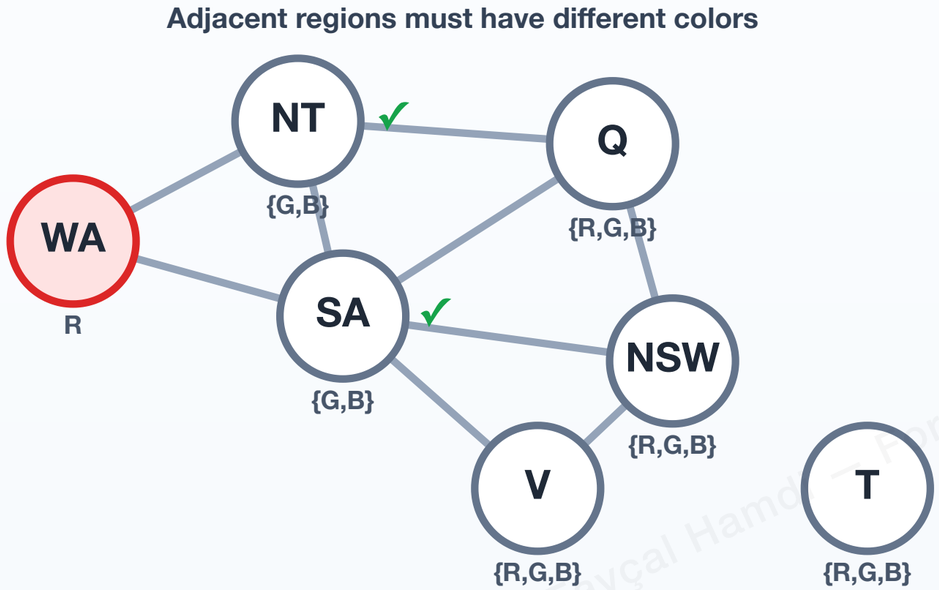
Result

Both remaining domains are still nonempty.

Legend

Assigned: R G B unassigned

Forward Checking in Action



Step 4 — continue or fail early

Main idea
Because every affected neighbor still has an available value, the branch remains viable and search can continue.

Current status
NT = {G,B}, SA = {G,B}

Continue when
Every checked domain remains nonempty.

Fail early when
If any neighboring domain becomes ∅, reject the branch and backtrack immediately.

Legend
Assigned: R G B unassigned

Constraint Propagation

Levels of local consistency

Constraint propagation repeatedly removes values that cannot participate in a consistent assignment.

Level	Scope	Consistency condition	Small example
Node consistency	One variable	Every remaining value satisfies all unary constraints on that variable.	$D(X) = \{1, 2, 3\}$ and $X \neq 3 \Rightarrow$ remove 3.
Arc consistency	Two variables	Every value of one variable has at least one supporting value in the neighboring domain.	For $X < Y$, an X value must have some larger value in $D(Y)$.
Path consistency	Three variables	A consistent assignment to two variables can be extended consistently through a third variable.	$X = Y, Y = Z, X \neq Z$ with all domains $\{1, 2\}$ is arc-consistent but path reasoning exposes the contradiction.

For path consistency, a consistent pair $(X_i = a, X_j = b)$ should have a value $c \in D(X_k)$ that is compatible with both assignments through the relevant constraints.

💡 In this lecture we study arc consistency in depth because it provides an effective balance between propagation strength and computational cost. AC-3 is the standard algorithmic example.

Arc Consistency

Every value must have support in neighboring domains

For a binary constraint between X_i and X_j, the arc $X_i \rightarrow X_j$ is consistent if:

$$\forall x \in D_i,\ \exists y \in D_j \text{ such that } (x, y) \text{ satisfies } C_{ij}$$

If a value x has no support in D_j, then x can be safely removed from D_i.

For the constraint $X < Y$, value $X = 3$ has no support if $D(Y) = \{1, 2\}$.

AC-3 Algorithm

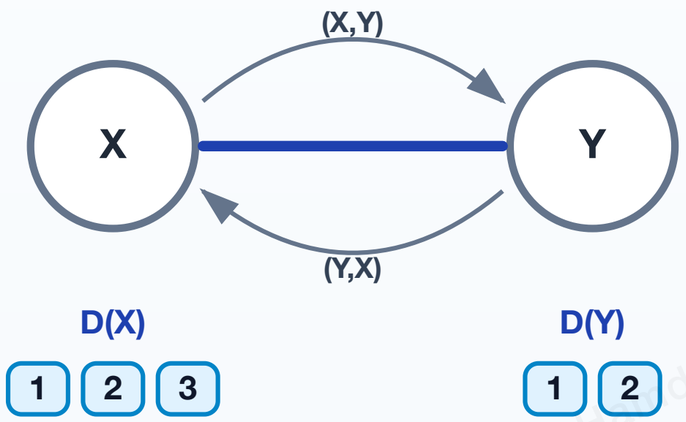
Enforce arc consistency with a queue of arcs

```
def AC3(csp):
    queue = all_arcs(csp)

    while queue is not empty:
        (Xi, Xj) = queue.pop()

        if revise(csp, Xi, Xj):
            if domain[Xi] is empty:
                return failure

            for each Xk in neighbors[Xi] except Xj:
                queue.add((Xk, Xi))

    return success
```


💡 Whenever a domain changes, neighboring arcs may need to be checked again.

AC-3 Walkthrough

Removing unsupported values from domains



Step 1 — initialize the queue

Main idea
AC-3 places both directed arcs in the queue. An arc (X,Y) asks whether every value of X has support in Y.

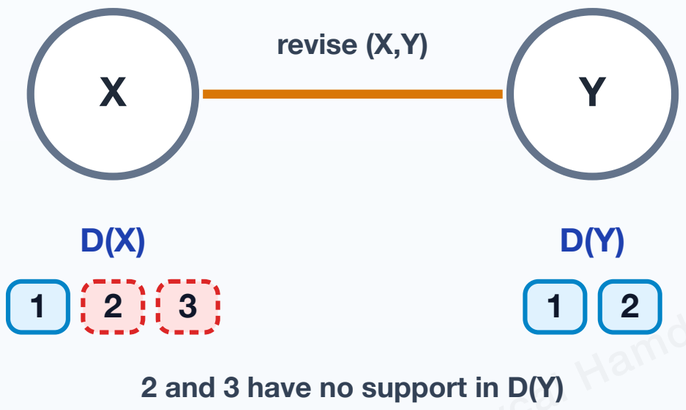
Domains
D(X)={1,2,3}, D(Y)={1,2}

Constraint
X < Y

Queue
[(X,Y), (Y,X)]

Next
Remove one arc and revise its first variable.

Removing unsupported values from domains



Step 2 — revise (X,Y)

Main idea
For each x ∈ D(X), AC-3 looks for some y ∈ D(Y) satisfying x < y.

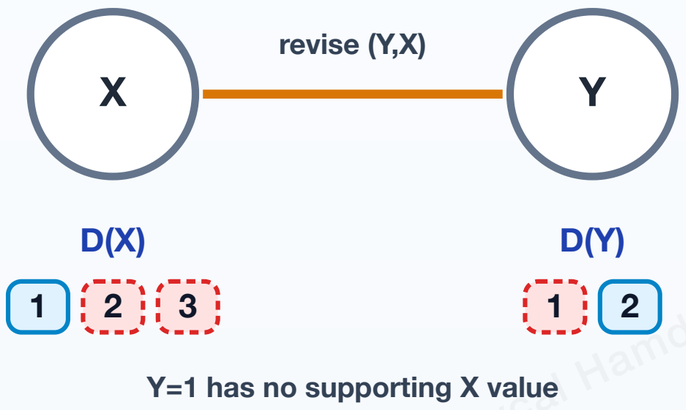
Support
X=1 is supported by Y=2.

No support
X=2 and X=3 have no larger value in D(Y).

Revise D(X)
{1,2,3} → {1}

Queue
[(Y,X)]

Removing unsupported values from domains



Step 3 — revise (Y,X)

Main idea
Now each $y \in D(Y)$ must have some $x \in D(X)$ satisfying the same constraint $x < y$.

Current D(X)

{1}

No support

Y=1 has no X value smaller than 1.

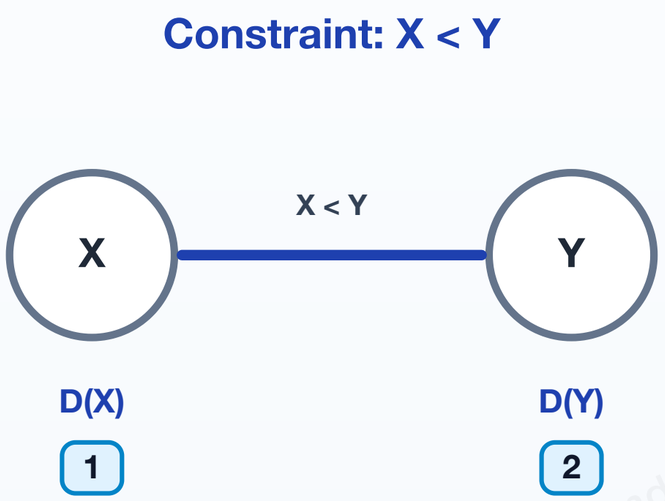
Revise D(Y)

{1,2} → {2}

Queue

∅

Removing unsupported values from domains



Step 4 — arc-consistent

Main idea
The queue is empty, so no remaining arc requires further revision. Every remaining value has support across the constraint.

Final domains
D(X)={1}, D(Y)={2}

Support check
X=1 is supported by Y=2, and Y=2 is supported by X=1.

Queue
∅

Result
The CSP is arc-consistent for this constraint.

Forward Checking, AC-3, and MAC

Same goal, different timing and scope

Method	When used	What it checks	Main role
Forward checking	After assigning a variable during search	Assigned variable against its unassigned neighbors	Detect immediate future failures cheaply
AC-3	As a propagation procedure	A queue of directed arcs until all relevant arcs are consistent	Enforce arc consistency
MAC	After each assignment during backtracking	Re-establish arc consistency in the remaining CSP	Combine systematic search with stronger propagation

MAC — Maintaining Arc Consistency means that after each search decision, arc consistency is restored before the search goes deeper.

💡 Do not confuse the two: **AC-3 is an algorithm for enforcing arc consistency; MAC is a search strategy that repeatedly invokes arc-consistency propagation during backtracking.**

Part III

CSP Heuristics, Local Repair, and Strategy Choice

Efficient CSP solving depends heavily on **which variable to assign next** and **which value to try first**.

Variable-Selection Heuristics

MRV chooses the most urgent variable; Degree breaks ties

Minimum Remaining Values — MRV

Choose the unassigned variable with the **fewest currently legal values**:

$$X^* = \arg\min_{X \text{ unassigned}} |D_{\text{current}}(X)|$$

Intuition: use the **fail-first** principle. If a variable is close to failure, discover that failure now rather than much deeper in the search tree.

Degree heuristic

If several variables tie under MRV, prefer the one constraining the largest number of other unassigned variables:

$$X^* = \arg\max_{X} |\{Y : \ Y \text{ is unassigned and constrained by } X\}|$$

Intuition: choose the variable that is most likely to affect the remaining problem.

> 💡 The usual policy is **MRV first, Degree as a tie-breaker**. Degree does not normally override a strictly better MRV choice.

MRV and Degree — Worked Choice

Two different questions: urgency first, influence second

Suppose the current unassigned variables have:

Variable	Legal values remaining	Unassigned neighbors
A	2	1
B	2	3
C	3	4
D	2	2

Step 1 — MRV

The minimum domain size is 2, so MRV gives a tie among: A, B, and D.

Step 2 — Degree tie-break

Among the tied variables, B constrains the most unassigned neighbors.

$$\boxed{B \text{ is selected}}$$

💡 C has the largest degree overall, but it is **not** selected because it did not tie for the best MRV value.

Value-Selection Heuristic

LCV chooses the value that leaves the most freedom

Once the variable X has been selected, the **Least Constraining Value (LCV)** heuristic orders its legal values by how little they restrict neighboring variables.

A useful score is:

$$\text{score}(X = v) = \sum_{Y \in N(X)} \left(|D_{\text{current}}(Y)| - |D(Y \mid X = v)| \right)$$

Choose the value with the **smallest** score.

Worked ordering

Candidate value	Neighbor values eliminated	LCV preference
Red	5	3rd
Green	2	**1st**
Blue	4	2nd

Therefore:

$$\boxed{X = \textit{Green} \text{ is tried first}}$$

💡 MRV and Degree answer **which variable?** LCV answers **which value first?** These heuristics change search order, not the definition of a solution.

MRV, Degree, and LCV Together

A common CSP search policy

```
def backtrack(assignment, csp):
    if assignment is complete:
        return assignment

    var = select_by_MRV_then_Degree(csp, assignment)

    for value in order_by_LCV(var, csp, assignment):
        if consistent(var, value, assignment):
            assign(var, value)
            inferences = forward_check_or_MAC(csp, var, value)
            if inferences do not fail:
                result = backtrack(assignment, csp)
                if result != failure:
                    return result
            undo_assignment_and_inferences()

    return failure
```


💡 A useful mental model is:

1. **MRV** — choose the most urgent variable.
2. **Degree** — break an MRV tie using the most influential variable.
3. **LCV** — try the least damaging value first.
4. **Forward checking or MAC** — infer consequences before recursing.

This separates **search control** from **constraint inference**.

Min-Conflicts Search

Repair a complete assignment

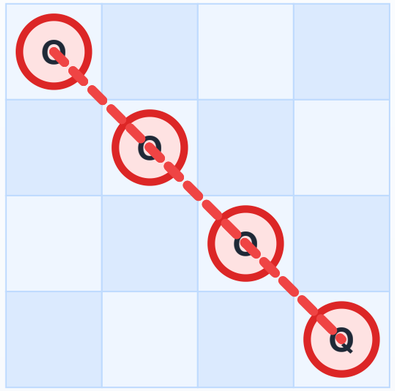
Min-conflicts is a local-search strategy for CSPs:

1. Start with a complete assignment, even if inconsistent.
2. Select a variable involved in at least one conflict.
3. Change its value to minimize the number of violated constraints.
4. Repeat until no conflicts remain or a step limit is reached.

> Min-conflicts is especially effective for large CSPs where solutions are dense and local repair is easier than systematic enumeration.

Min-Conflicts Walkthrough

Repairing a 4-Queens assignment



One queen per column; move a conflicted queen within its column.

Step 1 — start with a complete assignment

Main idea
Min-conflicts starts with a complete assignment, even when several constraints are violated.

Rows by column

[1,2,3,4]

Conflicts

All four queens are conflicted.

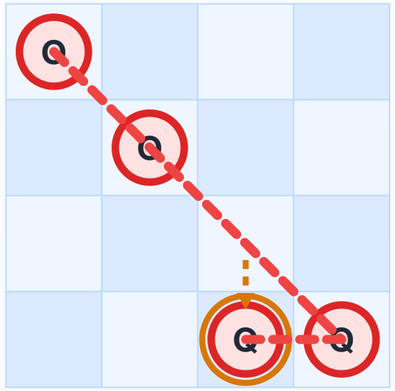
Action

Select a conflicted queen and move it to a minimum-conflict row.

Legend

conflicted queen | conflict-free queen

Repairing a 4-Queens assignment



Move column 3: row 3 → row 4

Step 2 — repair column 3

Main idea
Select the conflicted queen in column 3 and move it to a row with the fewest conflicts.

Before

[1,2,3,4]

Chosen move

Column 3: row 3 → row 4

Minimum conflicts

Row 4 gives 1 conflict; row 2 is tied with the same minimum.

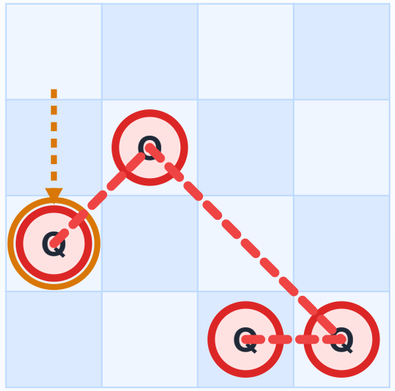
New state

[1,2,4,4]

Legend

conflicted queen | conflict-free queen

Repairing a 4-Queens assignment



Move column 1: row 1 → row 3

Step 3 — repair column 1

Main idea
The assignment is still inconsistent, so select another conflicted queen and again choose a minimum-conflict row.

Before

[1,2,4,4]

Chosen move

Column 1: row 1 → row 3

Minimum conflicts

Row 3 gives the minimum: 1 conflict.

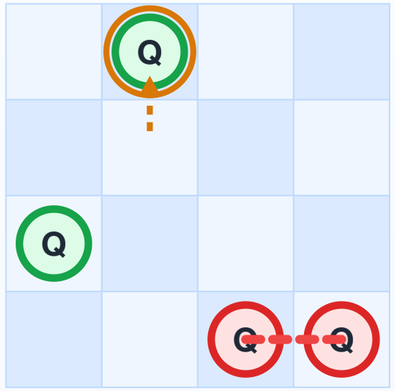
New state

[3,2,4,4]

Legend

conflicted queen | conflict-free queen

Repairing a 4-Queens assignment



Move column 2: row 2 → row 1

Step 4 — reduce to one conflict

Main idea
Move the conflicted queen in column 2 to row 1. It now has zero conflicts.

Before

[3,2,4,4]

Chosen move

Column 2: row 2 → row 1

Minimum conflicts

Row 1 gives 0 conflicts for this queen.

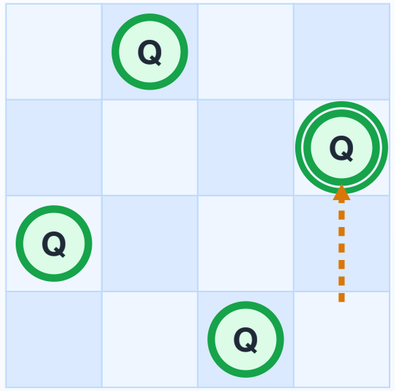
New state

[3,1,4,4]; only columns 3 and 4 conflict.

Legend

conflicted queen | conflict-free queen

Repairing a 4-Queens assignment



Move column 4: row 4 → row 2 — no conflicts remain

Step 5 — conflict-free assignment

Main idea
Move the remaining conflicted queen in column 4 to a zero-conflict row. No queens now attack each other.

Before

[3,1,4,4]

Final move

Column 4: row 4 → row 2

Solution

[3,1,4,2]

Result

All constraints are satisfied; stop.

Legend

conflicted queen | conflict-free queen

Min-Conflicts — Pseudocode

Local search over complete assignments

```
def min_conflicts(csp, max_steps):
    current = random_complete_assignment(csp)

    for step in range(max_steps):
        if current satisfies all constraints:
            return current

        var = random_conflicted_variable(current, csp)
        value = value_that_minimizes_conflicts(var, current, csp)
        current[var] = value

    return failure
```


Min-conflicts is not a complete algorithm: failure after a step limit does not prove that no solution exists.

Exploiting CSP Structure

The constraint graph can reduce computational difficulty

Structure	Solver idea	Why it helps
Disconnected components	Solve each connected component independently.	Independent subproblems do not need to be searched together.
Tree-structured CSP	Propagate constraints in one direction, then assign variables in a compatible order.	No cycles means decisions do not create complex feedback loops.
Nearly tree-structured CSP	Assign a small set of variables whose removal leaves a tree; then solve the remaining tree CSP.	A difficult cyclic problem can be reduced to several easier tree problems.
Dense constraint graph	Use strong propagation and good ordering, but expect many interactions.	More constraints may prune strongly, yet each decision can affect many variables.

For a tree-structured binary CSP with n variables and maximum domain size d, a standard directional arc-consistency procedure can solve the problem in $O(nd^2)$ time.

💡 The key lesson is structural: the same number of variables and domain values can be much easier or harder depending on the **shape of the constraint graph**.

CSP Strategy Comparison

Different methods serve different needs

Method	Main mechanism	Strength	Limitation
Backtracking	Depth-first assignment search	Simple and complete for finite CSPs	Can be slow without heuristics
Forward checking	Removes values from neighbors	Detects immediate failures	Limited lookahead
AC-3	Enforces arc consistency	Stronger pruning	May not solve the CSP alone
MAC	Maintains arc consistency during search	Powerful pruning	Higher per-node cost
Min-conflicts	Repairs complete assignments	Often very fast on large CSPs	Not complete

Worked Activity

Formulate and solve a small CSP

Three exams A, B, and C must be scheduled in two slots 1 and 2.

Constraints:

- $A \neq B$ because some students take both.
- $B \neq C$ because some students take both.
- A cannot be in slot 2.

Tasks:

1. Define X, D, and C.
2. Draw the binary constraint graph.
3. Apply **node consistency** to the unary constraint on A.
4. After node consistency, which variable would **MRV** select first?
5. If $A = 1$ is assigned, apply **forward checking**. What happens to $D(B)$?
6. Starting immediately after node consistency, apply **AC-3**. How far can propagation go before any search decision is required?
7. Give the final solution.

Expected insight:

After node consistency, $D(A) = \{1\}$ and $D(B) = D(C) = \{1, 2\}$

So MRV selects A.

With $A = 1$, forward checking forces $D(B) = \{2\}$.

After assigning $B = 2$, it forces $D(C) = \{1\}$.

AC-3 can propagate these consequences **before search**: the arc $B \rightarrow A$ removes 1 from $D(B)$, and then $C \rightarrow B$ removes 2 from $D(C)$.

Therefore the solution is:

$$\boxed{A = 1,\ B = 2,\ C = 1}$$

This example shows why stronger propagation can reduce or even eliminate search.

Mini-Quiz

Check the distinctions, not only the vocabulary

1. What is the difference between a **binary** constraint and a **global** constraint?
2. How does **node consistency** differ from **arc consistency**?
3. Why can arc consistency detect some failures that **forward checking** misses?
4. If two variables tie under MRV, what does the **Degree heuristic** do?
5. What question does **LCV** answer?
6. What is the difference between **AC-3** and **MAC**?
7. Why does failure of min-conflicts after a step limit not prove that the CSP is unsatisfiable?

Answers:

1. Binary refers to arity two; a global constraint represents a larger semantic relation often handled with specialized propagation.
2. Node consistency checks unary constraints; arc consistency requires support across a binary constraint.
3. Arc consistency can propagate between unassigned variables; forward checking propagates from the latest assignment.
4. Choose the tied variable constraining the most unassigned neighbors.
5. Which legal value should be tried first so that neighboring domains lose the fewest options.
6. AC-3 enforces arc consistency; MAC maintains arc consistency repeatedly during backtracking.
7. Min-conflicts is not complete, so reaching the step limit is only a search failure.

Lecture 4 Summary

Key ideas to retain

Representation and inference

- A CSP is defined by variables, domains, and constraints.
- Constraint arity and constraint semantics are distinct ideas; global constraints may carry exploitable structure.
- **Node, arc, and path consistency** represent progressively broader forms of local reasoning.
- Arc consistency is the main practical focus here because it supports strong propagation through **AC-3** and **MAC**.

Search and heuristics

- Backtracking searches systematically over partial assignments.
- Forward checking performs limited lookahead; MAC maintains stronger arc-consistency reasoning during search.
- **MRV chooses the variable**, **Degree breaks MRV ties**, and **LCV orders values**.
- Min-conflicts repairs complete assignments using local search.
- Constraint-graph structure can make a CSP substantially easier or harder.

End-of-Lecture Check

Five questions to test your understanding

1. Why are CSPs not just ordinary state-space search problems?
2. How does a constraint graph help a solver?
3. Why can arc consistency detect a contradiction between two unassigned variables that forward checking may not detect yet?
4. Why is **Degree** normally used after an **MRV tie** rather than as a replacement for MRV?
5. What is the conceptual difference between **AC-3**, **MAC**, and **min-conflicts**?

Preparation for next lecture: Review minimax decision making, utility values, and the difference between single-agent search and multi-agent adversarial search. Lecture 5 studies game playing and adversarial search.

Suggested Reading

Primary textbook

S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th edition, Prentice Hall, 2020.

- Chapter 6: Constraint Satisfaction Problems
- Focus especially on CSP formulation, constraint propagation, backtracking search, variable/value heuristics, and local search for CSPs.

> 💡 When reading, pay close attention to how CSP algorithms combine **search** and **inference**.